

TCP/UDP processing

The TCP buffer size is:

```
net.ipv4.tcp_rmem
net.ipv4.tcp_wmem
```

These values are an array of three integers specifying the minimum, average and maximum size of the TCP read and send buffers, respectively.

 **Note:** Values are in pages. To see the page size, use the command `$getconf PAGE_SIZE`

For the latest kernel (after 2.6), there is a feature for auto tuning, which dynamically adjusts the TCP buffer size till the maximum value is attained. This feature is turned on by default and I recommend that it be left turned on. You can check it by running the following command:

```
$cat /proc/sys/net/ipv4/tcp_moderate_rcvbuf
```

Then, to turn it on in case it is off, use the command given below:

```
$sysctl -w net.ipv4.tcp_moderate_rcvbuf=1
```

This setting allocates space up to the maximum value, in case you need to increase the maximum buffer size if you find the kernel buffer is your bottleneck. The average value need not be changed, but the maximum value will have to be set to higher than the BDP (bandwidth delay product) for maximum throughput.

$BDP = \text{Bandwidth (B/sec)} * \text{RTT (seconds)}$, where RTT (round trip time) can be calculated by pinging to any other system and finding the average time in seconds. You can change the value with `sysctl` by using the following command:

```
$sysctl -w net.ipv4.tcp_rmem="65535 131072 4194304"
$sysctl -w net.ipv4.tcp_wmem="65535 131072 194304"
```

Maximum pending connections

An application can specify the maximum number of pending requests to put in queue before processing one connection. When this value reaches the maximum, further connections start to drop out. For applications like a Web server, which issue lots of connections, this value has to be high for these to work properly. To see the maximum connection backlog, run the following command:

```
$ sysctl net.core.somaxconn
The default value is 128 and can increase to much secure value.
$sysctl -w net.core.somaxconn=2048
```

TCP timestamp

TCP timestamp is a TCP feature that puts a timestamp header in every packet to calculate the precise round trip time. This feature causes a little overhead and can be disabled, if it is not necessary to increase CPU performance to process packets.

```
$sysctl -w net.ipv4.tcp_timestamps=0
```

TCP SACK

TCP Selective Acknowledgements (SACK) is a feature that allows TCP to send ACK for every segment stream of packets, as compared to the traditional TCP that sends ACK for contiguous segments only. This feature can cause a little CPU overhead; hence, disabling it may increase network throughput.

```
# sysctl -w net.ipv4.tcp_sack=0
```

TCP FIN timeout

In a TCP connection, both sides have to close the connection independently. Linux TCP sends a FIN packet to close the connection and waits for FINACK till the defined time mentioned in...

```
net.ipv4.tcp_fin_timeout.
```

The default value (60) is quite high, and can be decreased to 20 or 30 to let the TCP close the connection and free resources for another connection.

```
$sysctl -w net.ipv4.tcp_fin_timeout=20
```

UDP buffer size

UDP generally doesn't need tweaks to improve performance, but you can increase the UDP buffer size if UDP packet losses are happening.

```
$ sysctl net.core.rmem_max
```

Miscellaneous tweaks

IP ports: `net.ipv4.ip_local_port_range` shows all the ports available for a new connection. If no port is free, the connection gets cancelled. Increasing this value helps to prevent this problem.

To check the port range, use the command given below:

```
$sysctl net.ipv4.ip_local_port_range
```

You can increase the value by using the following command:

```
$sysctl -w net.ipv4.ip_local_port_range='20000 60000'
```

Tuned

Tuned is a very useful tool to auto-configure your system according to different profiles. Besides doing manual tuning,